

## 分治法

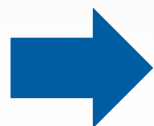
---

讲解人：陆伟

September 2, 2025

# 目录

## CONTENTS



1

分治法基本思想

2

分治法适用条件

3

分治法求解问题基本步骤

4

分治法复杂性分析

5

分治法求解问题典型案例

6

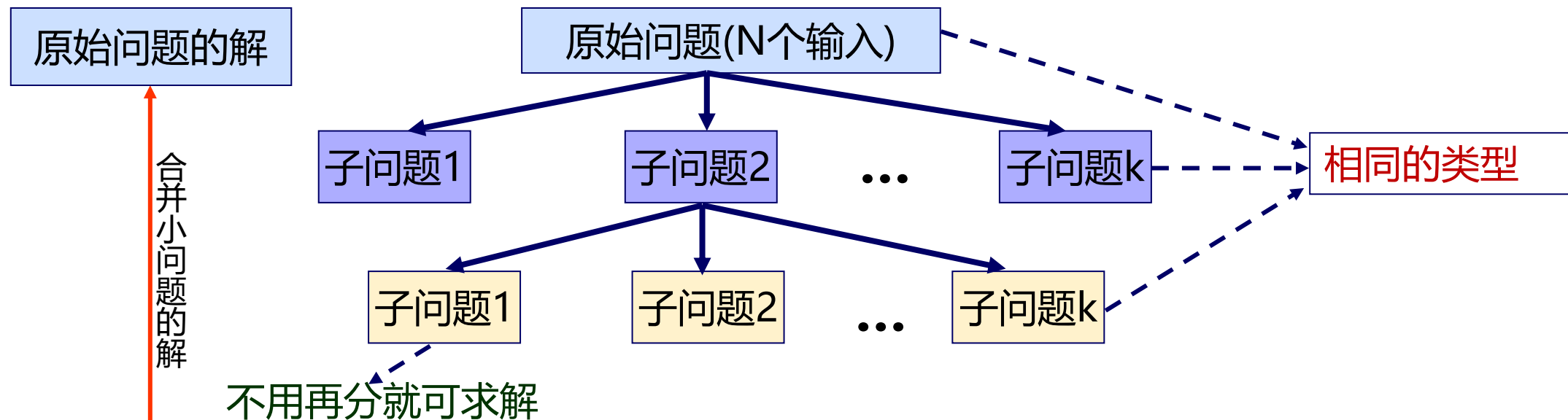
开放性讨论

7

总结

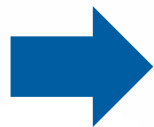
## 分治法基本思想

- 求解问题算法的复杂性一般都与问题规模相关，问题规模越小越容易处理。
- 分治法的基本思想是，将一个难以直接解决的大问题，分解为规模较小的相同子问题，直至这些子问题容易直接求解，并且可以利用这些子问题的解求出原问题的解。各个击破，分而治之。
- 分治法产生的子问题一般是原问题的较小模式，这就为使用递归技术提供了方便。**递归是分治法中最常用的技术。**



# 目录

## CONTENTS



1

分治法基本思想

2

分治法适用条件

3

分治法求解问题基本步骤

4

分治法复杂性分析

5

分治法求解问题典型案例

6

开放性讨论

7

总结

# 分治法适用条件

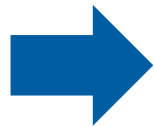
分治法所能解决的问题一般具有以下几个特征：

- 该问题的规模缩小到一定的程度就可以容易地解决；
- 该问题可以分解为若干个规模较小的相同问题；
- 利用该问题分解出的子问题的解可以合并为该问题的解；
- 该问题所分解出的各个子问题是相互独立的，即子问题之间不包含公共的子问题。

其中，第四条特征涉及到分治法的效率，如果各子问题是不独立的，则分治法要做许多不必要的工作，重复地解公共的子问题，此时虽然也可用分治法，但一般用**动态规划**较好。

# 目录

## CONTENTS



1

分治法基本思想

2

分治法适用条件

3

分治法求解问题基本步骤

4

分治法复杂性分析

5

分治法求解问题典型案例

6

开放性讨论

7

总结

## 分治法求解问题基本步骤

- 一般来说，分治法的求解过程由以下三个阶段组成：
  - **划分**：既然是分治，当然需要把规模为 $n$ 的原问题划分为 $k$ 个规模较小的子问题，并尽量使这 $k$ 个子问题的规模大致相同。
  - **求解子问题**：各子问题的解法与原问题的解法通常是相同的，可以用递归的方法求解各个子问题，有时递归处理也可以用循环来实现。
  - **合并**：把各个子问题的解合并起来，合并的代价因情况不同有很大差异，分治算法的有效性很大程度上依赖于合并的实现。

```
divide-and-conquer(P){  
    if ( | P | <= n0) adhoc(P); //解决小规模的问题  
    divide P into smaller subinstances P1,P2,...,Pk; //分解问题  
    for (i=1; i<=k; i++)  
        yi=divide-and-conquer(Pi); //递归的解各子问题  
    return merge(y1,...,yk); //将各子问题的解合并为原问题的解  
}
```

# 分治法复杂性分析

- 分治法复杂性分析基本依据 – 递推方程
- 两类常见的递推方程

$$T(n) = \begin{cases} O(1) & n = 1 \\ \sum_{i=1}^k a_i T(n-i) + f(n) & n > 1 \end{cases}$$

$$T(n) = \begin{cases} O(1) & n = 1 \\ aT(\frac{n}{b}) + f(n) & n > 1 \end{cases}$$

- 递推方程求解常见方法
  - 迭代
  - 递归树



# 分治法复杂性分析

## □ 典型递推方程求解

$$T(n) = \begin{cases} O(1) & n = 1 \\ aT(\frac{n}{b}) + f(n) & n > 1 \end{cases}$$

$$T(n) = n^{\log_b a} + \sum_{j=0}^{\log_b n - 1} a^j f(n / b^j)$$

**注意：**递归方程及其解只给出n等于b的方幂时T(n)的值，但是如果认为T(n)足够平滑，那么由n等于b的方幂时T(n)的值可以估计T(n)的增长速度。通常假定T(n)是单调上升的，从而当 $b^i \leq n < b^{i+1}$ 时， $T(b^i) \leq T(n) < T(b^{i+1})$ 。

# 分治法求解问题典型案例

例

## □ 排列问题

问题：R是由n个元素构成的序列集合， $R=\{r_1, r_2, \dots, r_n\}$ ，求R的全排列perm(R)。

- 1、理解问题
- 2、选择策略
- 3、算法设计
- 4、正确性证明
- 5、算法分析
- 6、程序设计

# 分治法求解问题典型案例

## □ 排列问题

### □ 理解问题

(1) 若 $R$ 中只有1个元素 $\{r\}$ ，则 $\text{perm}(R)=(r)$

(2) 若 $R$ 中只有2个元素 $\{r_1, r_2\}$ ，则

$$\text{perm}(R)=(r_1)\text{perm}(R_1)\cup(r_2)\text{perm}(R_2)$$

其中， $R_i=R-\{r_i\}$

(3) 若 $R$ 中有3个元素 $\{r_1, r_2, r_3\}$ ，则

$$\text{perm}(R)=(r_1)\text{perm}(R_1)\cup(r_2)\text{perm}(R_2)\cup(r_3)\text{perm}(R_3)$$

$$\text{perm}(R) = \begin{cases} (r) & n = 1 \\ \cup (r_i) \text{perm}(R_i) & n > 1 \end{cases}$$

# 分治法求解问题典型案例

## □ 排列问题

### □ 算法设计

#### 思想：分治法

依次将待排列的数组的后n-1个元素与第一个元素交换,则每次递归处理的都是后n-1个元素的全排列。当数组元素仅有一个时为此递归算法的出口。

### □ 算法分析

$$T(n) = \begin{cases} O(1) & n = 1 \\ nT(n-1) + O(n) & n > 1 \end{cases}$$

### □ 程序设计

- 编码
- 测试

练习

接口： perm(Type list[], int k, int m)

算法 perm(Type list[], int k, int m)

//生成列表list的全排列

//输入： 一个全排列元素列表list[0..*n*-1]

//输出： list的全排列集合

if k == m

for i ← 0 to m do

输出list[i]

else

for i ← k to m do

swap list[k] and list[i]

perm(list, k+1, m)

swap list[k] and list[i]

参照代码[Perm.java](#)

# 分治法求解问题典型案例

例

## □ 整数划分问题

问题：将给定正整数 $n$ 表示成一系列正整数之和  
 $n=n_1+n_2+\dots+n_k$ ，其中 $n_1\geq n_2\geq\dots\geq n_k\geq 1$ ， $k\geq 1$ 。  
求正整数 $n$ 的不同划分个数 $p(n)$ 。

例如正整数6有如下11种不同的划分：

6;

5+1;

4+2, 4+1+1;

3+3, 3+2+1, 3+1+1+1;

2+2+2, 2+2+1+1, 2+1+1+1+1;

1+1+1+1+1+1;

讨论与  
理解

思想



分治？

# 分治法求解问题典型案例

例

## □ 整数划分问题

有些问题本身都具有比较明显的递归关系，因而容易用递归函数直接求解。而有些问题递归关系却不明显。

在本例中，如果设 $p(n)$ 为正整数 $n$ 的划分数，则难以找到递归关系，因此考虑增加一个自变量：将最大加数 $n_1$ 不大于 $m$ 的划分个数记作 $q(n,m)$ 。可以建立 $q(n,m)$ 的如下递归关系。

$$q(n,m) = \begin{cases} 1 & n=1, m=1 \\ q(n,n) & m > n \\ 1 + q(n, n-1) & m = n \\ q(n, m-1) + q(n-m, m) & 1 < m < n \end{cases}$$



```
int q(int n, int m) {  
    if ((n < 1) || (m < 1)) return 0;  
    if ((n == 1) || (m == 1)) return 1;  
    if (n < m) return q(n,n);  
    if (n==m) return q(n, m-1) + 1;  
    return q(n, m-1) + q(n-m, m);  
}
```

思考：是否有重复计算问题？

扩展：水果摆盘问题？

扩展思考与讨论

# 分治法求解问题典型案例

例

## □ 顺序表搜索问题

问题：给定已按升序排好序的n个元素a[0:n-1]，现要在这n个元素中找出一特定元素x。

给定顺序序列：3，5，7，8，9，12，15  
搜索9

- ✓ 该问题的规模缩小到一定的程度就可以容易地解决；
- ✓ 该问题可以分解为若干个规模较小的相同问题；
- ✓ 分解出的子问题的解可以合并为原问题的解；
- ✓ 分解出的各个子问题是相互独立的。

编码



测试

$T(n)=T(n/2)+O(1)$   
 $T(n)= O(\log n)$

参照代码[Search.java](#)

递归实现

?

非递归实现

问题：如果要查找下标最小的x，如何做？

# 分治法求解问题典型案例

例

## □ 大整数乘法问题

问题：设X和Y都是n位二进制数，设计一个有效算法计算它们的乘积XY。

直接加运算： $O(n^2)$ ，能够做的更好？

分治法：

$$X = AB \quad Y = CD$$

$$X = A 2^{n/2} + B \quad Y = C 2^{n/2} + D$$

$$XY = AC 2^n + (AD+BC) 2^{n/2} + BD$$

$$T(n) = \begin{cases} O(1) & n = 1 \\ 4T(n/2) + O(n) & n > 1 \end{cases}$$

$$T(n) = O(n^{\log_4 n}) = O(n^2) \quad \text{✗ 没有改进}$$



# 分治法求解问题典型案例

例

## □ 大整数乘法问题

改进思想：通过代数变换，减少乘法次数。

$$1. XY = AC 2^n + ((A-B)(D-C)+AC+BD) 2^{n/2} + BD$$

$$T(n) = \begin{cases} O(1) & n = 1 \\ 3T(n/2) + O(n) & n > 1 \end{cases}$$

$$T(n) = O(n^{\log 3}) = O(n^{1.59})$$

✓ 较大的改进

- 如果将大整数分成更多段，用更复杂的方式把它们组合起来，将有可能得到更优的算法。
- 最终这个思想导致了快速傅利叶变换(Fast Fourier Transform)的产生。该方法也可以看作是一个复杂的分治算法。

## □ 矩阵乘法问题

$$\left. \begin{array}{l} A = [a_{ij}], B = [b_{ij}] \\ C = [c_{ij}] = A \cdot B \end{array} \right\} i, j = 1, 2, \dots, n.$$
$$\begin{bmatrix} c_{11} & c_{12} & \cdots & c_{1n} \\ c_{21} & c_{22} & \cdots & c_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ c_{n1} & c_{n2} & \cdots & c_{nn} \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix} \cdot \begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1n} \\ b_{21} & b_{22} & \cdots & b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ b_{n1} & b_{n2} & \cdots & b_{nn} \end{bmatrix}$$

问题归结为计算元素C[i,j]

$$c_{ij} = \sum_{k=1}^n a_{ik} \cdot b_{kj}$$

传统方法:

依据定义来计算A和B的乘积矩阵C, 则每计算C的一个元素C[i][j], 需要做n次乘法和n-1次加法。因此, 算出矩阵C的  $n^2$  个元素所需的计算时间为  $O(n^3)$

# 分治法求解问题典型案例

例

## □ 矩阵乘法问题—分治

问题足够小情况:

如果 $n=2$ , 则2个2阶方阵的乘积可以直接计算出来, 共需要8次乘法和4次加法。

分治思想: 将矩阵A, B和C中每一矩阵都分块成4个大小相等的子矩阵。由此可将方程 $C=AB$ 重写为:

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} \quad C_{12} = A_{11}B_{12} + A_{12}B_{22}$$

计算2个 $n$ 阶方阵的乘积可以转化为计算8个 $n/2$ 阶方阵的乘积和4个 $n/2$ 阶方阵的加法。加法可以在 $O(n^2)$ 时间内完成。

$$T(n) = \begin{cases} O(1) & n = 2 \\ 8T(n/2) + O(n^2) & n > 2 \end{cases}$$

# 分治法求解问题典型案例

例

## □ 矩阵乘法问题—Strassen

改进思想：若想降低时间复杂度，必须想办法减少乘法次数。即减小递推函数中的a，减小1也会有改进。

问题关键在于：计算2个2阶方阵的乘积时，能否用少于8次的乘积运算。

$$M_1 = A_{11}(B_{12} - B_{22})$$

$$M_2 = (A_{11} + A_{12})B_{22}$$

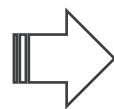
$$M_3 = (A_{21} + A_{22})B_{11}$$

$$M_4 = A_{22}(B_{21} - B_{11})$$

$$M_5 = (A_{11} + A_{22})(B_{11} + B_{22})$$

$$M_6 = (A_{12} - A_{22})(B_{21} + B_{22})$$

$$M_7 = (A_{11} - A_{21})(B_{11} + B_{12})$$



$$C_{11} = M_5 + M_4 - M_2 + M_6$$

$$C_{12} = M_1 + M_2$$

$$C_{21} = M_3 + M_4$$

$$C_{22} = M_5 + M_1 - M_3 - M_7$$

$$T(n) = \begin{cases} O(1) & n = 2 \\ 7T(n/2) + O(n^2) & n > 2 \end{cases}$$

$$T(n) = O(n^{\log_b a}) = O(n^{\log_2 7}) \approx O(n^{2.81})$$

✓较大的改进

Strassen的方法用了7次对n/2阶矩阵乘积的递归调用和18次n/2阶矩阵的加减运算。

## 分治法求解问题典型案例

例

### □ 矩阵乘法问题—进一步降低时间复杂度？

能否找到一种计算2阶方阵乘法的算法，使得乘法的计算次数少于7次？

Hopcroft和Kerr已经证明(1971)，计算2个  $2 \times 2$  矩阵的乘积，7次乘法是必要的。因此，要想进一步改进矩阵乘法的时间复杂性，就不能再基于计算  $2 \times 2$  矩阵的7次乘法这样的方法了。或许应当研究  $3 \times 3$  或  $5 \times 5$  矩阵的更好算法。

在Strassen之后又有许多算法改进了矩阵乘法的计算时间复杂性。目前最好的计算时间上界是  $O(n^{2.376})$ 。

目前所知道的矩阵乘法下界仍然是  $\Omega(n^2)$ ，但能否找到  $O(n^2)$  的算法？

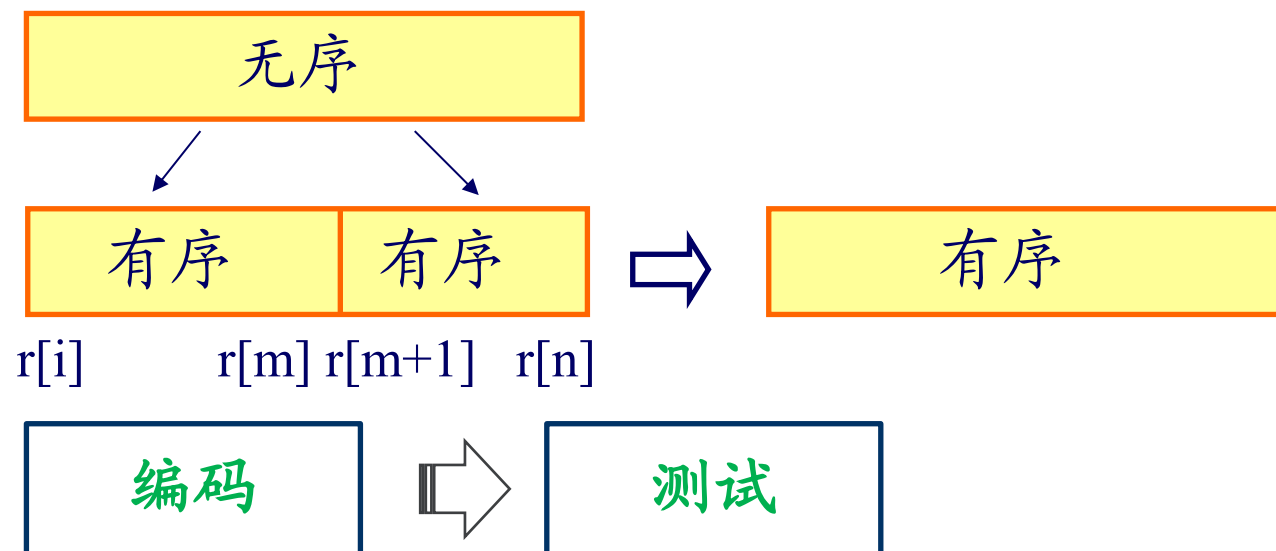
# 分治法求解问题典型案例

例

## □ 排序问题－合并

问题：给定一个可排序的 $n$ 个元素序列（数字、字符或字符串），对它们按照非降序方式重新排列。

基本思想：将待排序元素分成大小大致相同的2个子集合，分别对2个子集合进行排序，最终将排好序的子集合合并成为所要求的排好序的集合。



$$T(n) = \begin{cases} O(1) & n \leq 1 \\ 2T(n/2) + O(n) & n > 1 \end{cases}$$

$T(n)=O(n\log n)$  渐进意义下的最优算法  
辅助空间  $O(n)$

# 分治法求解问题典型案例

例

## □ 排序问题 – 合并 – 讨论

- 合并排序在最坏情况下的键值比较次数十分接近于任何基于比较的排序算法在理论上能够达到的最少次数
- 合并排序需要辅助的 $O(n)$ 空间
- 合并排序没能充分利用序列的特征，因此，最好情况下。。。
- 进一步优化：自然合并

# 分治法求解问题典型案例

例

## □ 排序问题—快速排序

问题：给定一个可排序的 $n$ 个元素序列（数字、字符或字符串），对它们按照非降序方式重新排列。

基本思想：对待排序数组不断拆分，拆分的同时不断交换元素，归位分裂点元素。记录的比较和交换是从两端向中间进行，记录每次移动的距离较大，因而总的比较和移动次数较少。

由Hoare(图灵奖获得者)1962年提出，被评为20世纪十大优秀算法之一。

不大于



不小于

编码



测试

关键在于分裂点（哨兵）选取和分区函数设计



# 分治法求解问题典型案例

例

## □ 第k大元素问题 – 快速排序思想延伸

问题：给定线性序集中 $n$ 个元素和一个整数 $k$ ， $1 \leq k \leq n$ ，要求找出这 $n$ 个元素中第 $k$ 小的元素。

基本思想：借鉴快速排序思想，如果将这 $n$ 个元素依其线性序排列时，排在第 $k$ 个位置的元素就是要找的元素。

```
template<class Type>
Type randomizedSelect(Type a[],int p,int r,int k) {
    if (p==r) return a[p];
    int i=randomizedPartition(a,p,r),
        j=i-p+1;
    if (k<=j) return randomizedSelect(a,p,i,k);
    else return randomizedSelect(a,i+1,r,k-j);
}
```

最坏时间复杂度：  $O(n^2)$

平均时间复杂度：  $O(n)$

最坏时间复杂度：  $O(n)$ ?

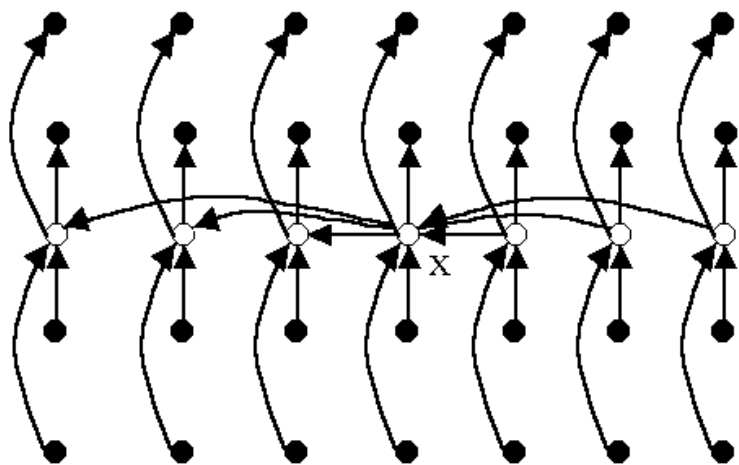
# 分治法求解问题典型案例

例

## □ 线性时间选择问题

如果能在线性时间内找到一个划分基准，使得按这个基准所划分出的2个子数组的长度都至少为原数组长度的 $\varepsilon$ 倍( $0 < \varepsilon < 1$ 是某个正常数)，那么就可以在最坏情况下用 $O(n)$ 时间完成选择任务。

例如，若 $\varepsilon = 9/10$ ，算法递归调用所产生的子数组的长度至少缩短 $1/10$ 。所以，在最坏情况下，算法所需的计算时间 $T(n)$ 满足递归式 $T(n) \leq T(9n/10) + O(n)$ 。由此可得 $T(n) = O(n)$ 。



# 分治法求解问题典型案例

例

## □ 最近点对问题

问题：给定平面上的 $n$ 个点，找其中的一对点，使得在 $n$ 个点组成的所有点对中，该点对之间的距离最小。

常规思想：只要将每一点与其它 $n-1$ 个点的距离算出，找出其中距离最小的两点即可。

问题：效率太低， $T(n)=O(n^2)$

分治思想：将平面上 $n$ 个点的集合 $S$ 划分为两个子集 $S_1$ 和 $S_2$ ，每个子集约 $n/2$ 个点，然后在每个子集中寻找最接近点对。

问题在于合并：如何根据子集中的最接近点对求得原集合中的最接近点对。

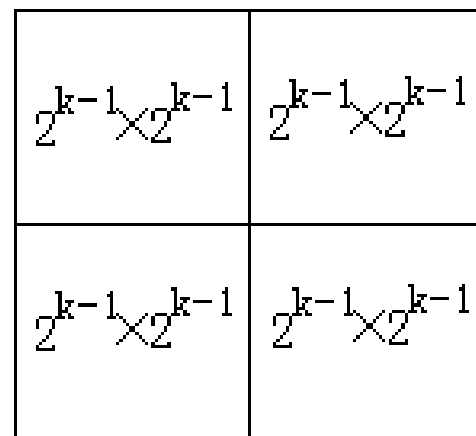
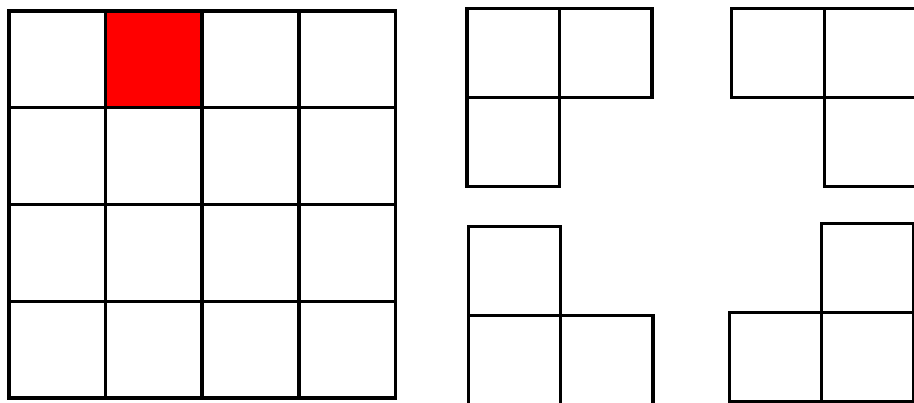
# 分治法求解问题典型案例

例

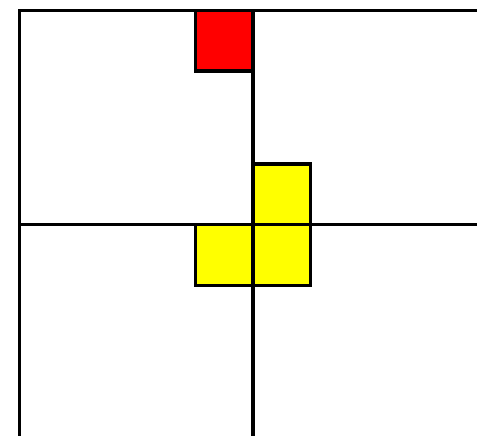
## □ 棋盘覆盖问题

在一个 $2^k \times 2^k$ 个方格组成的棋盘中，恰有一个方格与其它方格不同，称该方格为一特殊方格，且称该棋盘为一特殊棋盘。在棋盘覆盖问题中，要用图示的4种不同形态的L型骨牌覆盖给定的特殊棋盘上除特殊方格以外的所有方格，且任何2个L型骨牌不得重叠覆盖。

在任何一个 $2^k \times 2^k$ 的棋盘覆盖中，用到的L型骨牌个数恰为 $(4^k - 1)/3$



(a)



(b)

## 开放性讨论

---

- 应用分治法求解问题时，应该把问题分解为多少个子问题合适呢？
- 分治法一定可以提高解决问题的效率么？
  - 计算 $n$ 个数字 $a_0, a_1, \dots, a_{n-1}$ 的和。求 $n$ 个元素中的最大值。计算 $n$ 的阶乘。
  - 计算 $a$ 的 $n$ 次幂
- 分治法最终能够降低算法复杂性的本质
- 分治法降低算法复杂性常见措施
  - 代数变换，减少子问题个数
  - 利用中间结果
  - 数据预处理

## 关于本讲内容

---



**祝各位学习愉快!**

# 感谢观看！

讲解人：陆伟